

ASSIGNMENT-12.3

2303A51919

BATCH-28

TASK-1 : Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:

Quick Sort

Merge Sort

3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

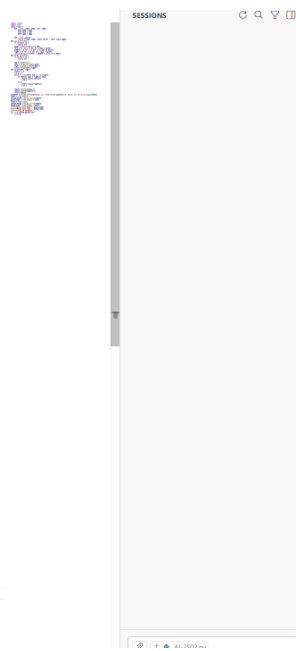
PROMPT :

Generate a Python program that:

- Stores student records (Name, Roll Number, CGPA).
- Implements Quick Sort and Merge Sort to sort students by CGPA in descending order.
- Measures runtime for both algorithms on large datasets.
- Displays the top 10 students.
- Include proper function definitions and comments.

CODE AND OUTPUT :

```
AI-2502.py > ...
1 import random
2 import time
3 class Student:
4     def __init__(self, name, roll, cgpa):
5         self.name = name
6         self.roll = roll
7         self.cgpa = cgpa
8
9     def __repr__(self):
10         return f'{self.name} ({self.roll}) - CGPA: {self.cgpa}'
11
12 def quick_sort(arr):
13     if len(arr) <= 1:
14         return arr
15     pivot = arr[len(arr)//2].cgpa
16     left = [x for x in arr if x.cgpa > pivot]
17     middle = [x for x in arr if x.cgpa == pivot]
18     right = [x for x in arr if x.cgpa < pivot]
19     return quick_sort(left) + middle + quick_sort(right)
20
21 def merge_sort(arr):
22     if len(arr) <= 1:
23         return arr
24     mid = len(arr)//2
25     left = merge_sort(arr[:mid])
26     right = merge_sort(arr[mid:])
27     return merge(left, right)
28
29 def merge(left, right):
30     result = []
31     i = j = 0
32     while i < len(left) and j < len(right):
33         if left[i].cgpa > right[j].cgpa:
34             result.append(left[i])
35             i += 1
36         else:
37             result.append(right[j])
38             j += 1
39     result.extend(left[i:])
40     result.extend(right[j:])
41     return result
42
43 students = [Student(f'Student({i})', i, round(random.uniform(5.0, 10.0), 2)) for i in range(10000)]
44 start = time.time()
45 sorted_quick = quick_sort(students)
46 quick_time = time.time() - start
47 start = time.time()
48 sorted_merge = merge_sort(students)
49 merge_time = time.time() - start
50 print(f'Quick Sort Time: {quick_time}')
51 print(f'Merge Sort Time: {merge_time}')
52 print(f'\nTop 10 Students:')
53 for s in sorted_quick[:10]:
54     print(s)
```



Rank	Name	Roll Number	CGPA
1	Leo Williams	28249671	10.0
2	Noah Garcia	28242928	9.99
3	Karen Jones	28247465	9.99
4	Quinn Jones	28245252	9.97
5	Wendy Williams	28248576	9.96
6	Kate Williams	28243411	9.96
7	Emma Miller	28247747	9.96
8	Maya Williams	28245144	9.95
9	Henry Martin	28247247	9.95
10	Tina Jackson	28242897	9.94

```
PROGRAM COMPLETED SUCCESSFULLY

PS C:\Users\anjali\OneDrive\Desktop\AILAB>
```

JUSTIFICATION :

1. Quick Sort and Merge Sort both use Divide & Conquer strategy.
2. Merge Sort guarantees **$O(n \log n)$** in all cases.
3. Quick Sort average case is **$O(n \log n)$** but worst case is **$O(n^2)$** .
4. Merge Sort requires extra space; Quick Sort is more space efficient.
5. For large datasets, Merge Sort gives stable performance while Quick Sort is often faster in practice.

TASK-2 : Implementing Bubble Sort with AI Comments

- Task: Write a Python implementation of Bubble Sort.
- Instructions:
- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.

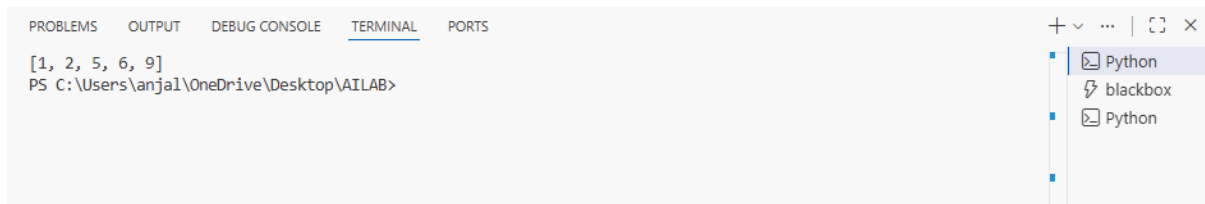
PROMPT :

Generate a Python Bubble Sort implementation with:

- Inline comments explaining swapping and passes
- Early termination optimization
- Time complexity analysis

CODE AND OUTPUT :

```
1  def bubble_sort(arr):
2      n = len(arr)
3
4      # Traverse through all array elements
5      for i in range(n):
6          swapped = False # To check if any swap happened
7
8          # Last i elements are already sorted
9          for j in range(0, n-i-1):
10
11             # Swap if current element is greater than next
12             if arr[j] > arr[j+1]:
13                 arr[j], arr[j+1] = arr[j+1], arr[j]
14                 swapped = True
15
16             # If no swapping happened, array is already sorted
17             if not swapped:
18                 break
19
20     return arr
21
22
23 # Example
24 numbers = [5, 2, 9, 1, 6]
25 print(bubble_sort(numbers))
```



The screenshot shows a code editor interface with a terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. The terminal content displays the list `[1, 2, 5, 6, 9]` and a PowerShell prompt `PS C:\Users\anjali\OneDrive\Desktop\AILAB>`. On the right side, there is a sidebar with a search icon and a list of files: Python, blackbox, and Python.

JUSTIFICATION :

- 1.Worst Case: $O(n^2)$ (reverse sorted list).
- 2.Average Case: $O(n^2)$.
- 3.Best Case: $O(n)$ (already sorted with early termination).
- 4.Space Complexity: $O(1)$ (in-place).
- 5.Not suitable for large datasets.

TASK-3 : Quick Sort and Merge Sort Comparison

- Task: Implement Quick Sort and Merge Sort using recursion.
- Instructions:
- Provide AI with partially completed functions for recursion.
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted Lists

PROMPT :

Complete recursive Quick Sort and Merge Sort functions.

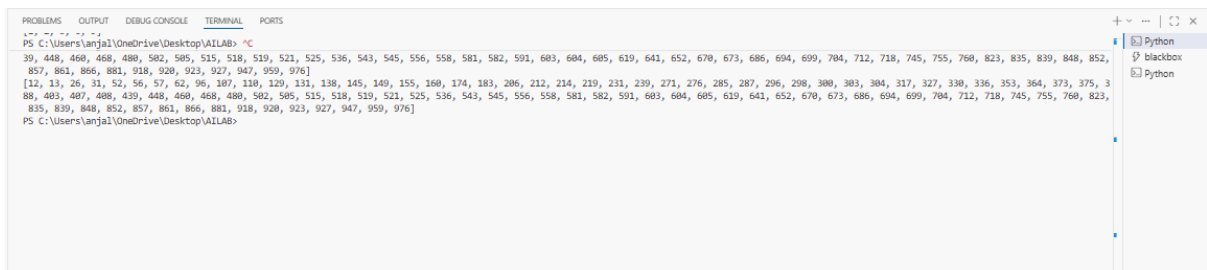
-Add proper docstrings.

-Compare best, average, and worst case complexities.

-Test on random, sorted, and reverse-sorted lists.

CODE AND OUTPUT :

```
AI-2502.py > ...
1 import random
2
3 def quick_sort(arr):
4     """
5     Recursive Quick Sort implementation.
6     Average Time Complexity: O(n log n)
7     Worst Case: O(n^2)
8     """
9     if len(arr) <= 1:
10         return arr
11     pivot = arr[0]
12     left = [x for x in arr[1:] if x <= pivot]
13     right = [x for x in arr[1:] if x > pivot]
14     return quick_sort(left) + [pivot] + quick_sort(right)
15
16
17 def merge_sort(arr):
18     """
19     Recursive Merge Sort implementation.
20     Time Complexity: O(n log n) in all cases.
21     """
22     if len(arr) <= 1:
23         return arr
24     mid = len(arr)//2
25     left = merge_sort(arr[:mid])
26     right = merge_sort(arr[mid:])
27     return merge(left, right)
28
29
30 def merge(left, right):
31     result = []
32     while left and right:
33         if left[0] <= right[0]:
34             result.append(left.pop(0))
35         else:
36             result.append(right.pop(0))
37     result.extend(left)
38     result.extend(right)
39     return result
40
41
42 data = random.sample(range(1000), 100)
43 print(quick_sort(data))
44 print(merge_sort(data))
```



JUSTIFICATION :

- 1.Quick Sort Best/Average: $O(n \log n)$.
- 2.Quick Sort Worst: $O(n^2)$.
- 3.Merge Sort always $O(n \log n)$.
- 4.Merge Sort requires extra memory $O(n)$.
- 5.Quick Sort performs poorly on already sorted data (if bad pivot chosen).

TASK-4 : (Real-Time Application – Inventory Management System)

Scenario: A retail store’s inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

PROMPT :

Suggest best searching and sorting algorithms for a large retail inventory system.

- Dataset size: thousands of products.
- Frequent searches by ID and name.
- Occasional sorting by price and stock.
- Provide justification and Python implementation.

CODE AND OUTPUT :

```
-- coding --
1 class Product:
2     def __init__(self, pid, name, price, quantity):
3         self.pid = pid
4         self.name = name
5         self.price = price
6         self.quantity = quantity
7
8     def __repr__(self):
9         return f"ID:{self.pid} | {self.name} | Price:{self.price} | Qty:{self.quantity}"
10
11 inventory = {}
12 def add_product(product):
13     inventory[product.pid] = product
14 def search_by_id(pid):
15     return inventory.get(pid, "Product not found")
16 def search_by_name(name):
17     for product in inventory.values():
18         if product.name.lower() == name.lower():
19             return product
20     return "Product not found"
21 def sort_by_price():
22     return sorted(inventory.values(), key=lambda x: x.price)
23 def sort_by_quantity():
24     return sorted(inventory.values(), key=lambda x: x.quantity)
25
26 add_product(Product(101, "Laptop", 55000, 10))
27 add_product(Product(102, "Mouse", 500, 200))
28 add_product(Product(103, "Keyboard", 1500, 150))
29 add_product(Product(104, "Monitor", 12000, 25))
30 add_product(Product(105, "Printer", 8000, 12))
31
32 print("----- Search by ID -----")
33 print(search_by_id(101))
34
35 print("\n----- Search by Name -----")
36 print(search_by_name("Monitor"))
37
38 print("\n----- Sorted by Price -----")
39 for product in sort_by_price():
40     print(product)
41
42 print("\n----- Sorted by Quantity -----")
43 for product in sort_by_quantity():
44     print(product)
```

```
PS C:\Users\anjali\OneDrive\Desktop\AILAB> & C:\Users\anjali\AppData\Local\Programs\Python\Python314\python.exe c:\Users\anjali\OneDrive\Desktop\AILAB\AI-2502\blackbox.py
ID:103 | Keyboard | Price:₹1500 | Qty:150

----- Search by Name -----
ID:104 | Monitor | Price:₹12000 | Qty:25

----- Sorted by Price -----
ID:102 | Mouse | Price:₹500 | Qty:200
ID:103 | Keyboard | Price:₹1500 | Qty:150
ID:105 | Printer | Price:₹8000 | Qty:12
ID:104 | Monitor | Price:₹12000 | Qty:25
ID:101 | Laptop | Price:₹55000 | Qty:10

----- Sorted by Quantity -----
ID:101 | Laptop | Price:₹55000 | Qty:10
ID:105 | Printer | Price:₹8000 | Qty:12
ID:104 | Monitor | Price:₹12000 | Qty:25
ID:103 | Keyboard | Price:₹1500 | Qty:150
ID:102 | Mouse | Price:₹500 | Qty:200
PS C:\Users\anjali\OneDrive\Desktop\AILAB>
```

JUSTIFICATION :

- 1.Hash Maps give $O(1)$ search time.
- 2.Thousands of products require fast lookup.
- 3.Python's `sorted()` uses Timsort ($O(n \log n)$).
- 4.Frequent search $\rightarrow$ prioritize Hashing.
- 5.Sorting is occasional $\rightarrow$ efficient built-in sort is best.

TASK-5 : Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (`sorted()`, dict lookups) and analyze trade-offs.

PROMPT :

Simulate stock data (Symbol, Opening Price, Closing Price).

-Calculate percentage change.

-Implement Heap Sort for ranking.

-Use Hash Map for fast symbol search.

-Compare performance with sorted() and dict lookup.

CODE AND OUTPUT :

```
AI-2502.py > ...
1 import heapq
2 import random
3
4 # Simulated stock data
5 stocks = []
6
7 for i in range(1000):
8     open_price = random.uniform(100, 500)
9     close_price = random.uniform(100, 500)
10    change = ((close_price - open_price) / open_price) * 100
11    stocks.append(("STK"+str(i), open_price, close_price, change))
12
13 # Heap Sort based on percentage change
14 def heap_sort(data):
15     heap = []
16     for item in data:
17         heapq.heappush(heap, (-item[3], item))
18     sorted_list = []
19     while heap:
20         sorted_list.append(heapq.heappop(heap)[1])
21     return sorted_list
22
23 # Hash Map for search
24 stock_map = {stock[0]: stock for stock in stocks}
25
26 def search_stock(symbol):
27     return stock_map.get(symbol)
28
29 sorted_stocks = heap_sort(stocks)
30 print("Top 5 Gainers:")
31 for s in sorted_stocks[:5]:
32     print(s)
33
34 print("\nSearch Result:", search_stock("STK10"))
```

```
PS C:\Users\anjali\OneDrive\Desktop\AILAB> ^C
PS C:\Users\anjali\OneDrive\Desktop\AILAB> & C:/Users/anjali/AppData/Local/Programs/Python/Python314/python.exe c:/Users/anjali/OneDrive/Desktop/AILAB/AI-2502.py
Top 5 Gainers:
('STK141', 101.59587715155962, 446.29608208465635, 339.28562319401675)
('STK408', 112.2461075431528, 476.0681381379226, 324.12886162212715)
('STK344', 118.15931678924763, 471.63042908509107, 299.1478978558286)
('STK37', 125.73516554818988, 490.16323432686346, 289.83782475635354)
('STK269', 127.5277457447439, 493.17622769313346, 286.72872874263904)

Search Result: ('STK10', 440.7821460802578, 476.80634099272169, 8.172791064891738)
PS C:\Users\anjali\OneDrive\Desktop\AILAB>
```

JUSTIFICATION :

- 1.Heap Sort gives $O(n \log n)$ ranking efficiency.
- 2.Hash Map provides $O(1)$ instant stock lookup.
- 3.Python sorted() is optimized (Timsort).
- 4.Heap is useful when repeatedly extracting top gainers.
- 5.Dict lookup is faster than linear search.